

pecepe.

Aplicativo de Tareo de Operaciones

Sistema web colaborativo de gestión de horas que automatiza el cálculo de TTHH, horas extras estratificadas y genera el reporte corporativo de operaciones con validación multi-supervisor en tiempo real.

■ Python 3.12 · ■ Streamlit · ■ PostgreSQL / Supabase · ■■ Render · ■ Plotly · ■ openpyxl · ■ SHA-256



Métricas del Proyecto

160 Registros por jornada	159/160 Coincidencia reporte real	6 Pestañas funcionales	7 Supervisores concurrentes
-------------------------------------	---	----------------------------------	---------------------------------------



Flujo de Datos — De la marcación al reporte final



Módulos de la Aplicación — 6 Pestañas

 1. Cargar Sube el xlsx del sistema. Auto-detecta cabecera, normaliza columnas y procesa hasta 160 registros. Botón reinicio para nueva jornada. ■ Solo Coordinador	 2. Condiciones / TTHH Editor masivo e individual por grupo. Corrido, Teórico 12h, Refrigerio, Descuento/Aumento extra, Jornada noche. Recalcula en tiempo real. ■ Coordinador + Supervisores
 3. Aprobación Supervisor revisa TTHH de su grupo y aprueba. Panel de avance. Solo lo aprobado pasa al reporte final. Aprobación masiva o individual. ■ Coordinador + Supervisores	 4. Reporte Final Genera REPORTE_OPERACIONES con 18 columnas formato corporativo. Descarga Excel (Calibri azul) y CSV. KPIs de horas totales, 25% y 35%. ■ Solo Coordinador
 Dashboard 7 gráficos Plotly interactivos con filtros. Sección A: tareo original (turnos, registros, condiciones, marcación). Sección B: TTHH, horas, aprobación. ■ Coordinador + Supervisores	 Compartir (WhatsApp) Resumen y reporte formateado con emojis. Botón verde abre WhatsApp listo para enviar. Descarga Excel adjuntable. Instrucciones export PNG. ■ Coordinador + Supervisores

Motor de Cálculo — Reglas RR.HH. (tareo_core.py)

Campo	Regla / Descripción		
Horas Marcación	SALIDA – ENTRADA (decimal). Cruce de medianoche: si $SALIDA \leq ENTRADA \rightarrow +24$ h automático		
Refrigerio (45 min)	Se descuenta SIEMPRE, SALVO: grupo en lista sin refrigerio (E, PCP, N, 5) · OBS = 'C' · OBS = '12'		
Teórico 12h	Si OBS = '12' $\rightarrow$ TTHH = 12.0 h exactas. Horario DÍA: 07:00–19:00. NOCHE: 19:00–07:00 o 20:00–08:00		
Corrido (C)	Sin descuento de refrigerio. Horario real de entrada/salida sin ajuste		
Descuento extra	–1, –2 o –3 horas adicionales restadas al bruto de marcación		
Aumento extra	+1, +2 o +3 horas adicionales sumadas al bruto de marcación		
TTHH final	$\max(0, HORAS_MARCACION - \text{refrigerio} - \text{descuento} + \text{aumento})$ · si Teórico12 $\rightarrow$ 12.0 h fijo		

Tramo	Rango	Color	Descripción
Hora Normal	0 $\rightarrow$ 8 h	■ Verde	Jornada base. $\min(\text{TTHH}, 8.0)$
Hora 25%	8 $\rightarrow$ 10 h	■ Ámbar	Extra tope. $\min(\max(\text{TTHH}-8, 0), 2.0)$
Hora 35%	> 10 h	■ Rojo	Extra alto. $\max(\text{TTHH}-10, 0)$
Hora 100%	dom/feriado	■ Morado	Manual — no calculado automáticamente
BONO HH	especial	■ Gris	Manual — ingreso posterior



Roles y Control de Acceso

■

Coordinador

donet · donet2026

- ✓ Cargar / reiniciar tareo del sistema
- ✓ Editar condiciones de TODOS los grupos
- ✓ Aprobar / desaprobar cualquier grupo
- ✓ Generar y descargar el reporte final
- ✓ Dashboard completo (todos los grupos)
- ✓ Compartir reporte final por WhatsApp
- ✓ Configurar reglas globales (sidebar)
- ✓ Gestión de usuarios — crear/editar/borrar

■

Supervisor de Grupo

sup_N · sup_E · sup_PCP · sup_1...5

- ✗ Cargar / reiniciar tareo
- ✓ Editar condiciones de su grupo
- ✓ Aprobar / desaprobar su grupo
- ✗ Reporte final completo
- ✓ Dashboard filtrado a su grupo
- ✓ Compartir resumen de su grupo WhatsApp
- ✗ Configurar reglas globales
- ✗ Gestión de usuarios

Arquitectura Técnica

Capa	Tecnología	Versión	Función
Lenguaje	Python	3.12	Base completa del proyecto
UI / Web	Streamlit	≥ 1.40	Dashboard responsivo — PC, móvil, tablet
Datos	pandas	≥ 2.0	DataFrames — carga, cálculo, exportación
Excel	openpyxl	≥ 3.1	Lectura/escritura xlsx con formato corporativo
Gráficos	Plotly	≥ 5.0	7 gráficos interactivos — descarga PNG nativa
ORM / DB	SQLAlchemy	≥ 2.0	Acceso PostgreSQL · NullPool para Supabase
Driver DB	psycopg2-binary	≥ 2.9	Conector PostgreSQL con soporte SSL
Base datos	Supabase	PG 17.6	PostgreSQL cloud · 3 tablas (usuarios/config/tareo)
Hosting	Render	Free	Deploy automático en push a main · Python 3.12.7
Autenticación	SHA-256 + salt	—	Contraseñas nunca en claro · sal 16 chars hex
VCS	Git / GitHub	—	Control de versiones · auto-deploy Render

■

Modo Online — Supabase

Con DATABASE_URL configurada → usa PostgreSQL en Supabase. Persistencia compartida entre todos los supervisores. Escritura por subconjunto (UPDATE WHERE id IN grupo). SSL requerido. Multi-usuario concurrente sin conflictos.

■

Modo Offline — Archivos locales

Sin DATABASE_URL → archivos locales: estado_tareo.pkl (DataFrame), usuarios.json (hashes SHA-256), config.json (reglas). Ideal para desarrollo local o trabajo sin conexión. Mismo código, cero configuración adicional.

Esquema de Base de Datos — Supabase / PostgreSQL 17.6

TABLE: usuarios usuario TEXT PK salt TEXT hash TEXT -- SHA-256 rol TEXT -- coord/sup grupo TEXT -- NULL coord	TABLE: config id INT PK = 1 data JSONB -- refrigerio_min -- grupos_sin_refrig -- jornadas, producto	TABLE: tareo id SERIAL PK NOMBRES TEXT GRUPO TEXT Corrido BOOL Aprobado BOOL ... +15 columnas
---	---	--

Tabla	Estrategia de escritura
usuarios	Upsert (INSERT ... ON CONFLICT DO UPDATE) — idempotente
config	Upsert sobre id=1 — fila única global editable desde el sidebar
tareo	TRUNCATE + INSERT al cargar · UPDATE WHERE id IN (grupo) al editar supervisor

Seguridad del Aplicativo

✓ Elemento	Implementación
■ Contraseñas	Hash SHA-256 con sal aleatoria de 16 chars hex. Nunca almacenadas en claro en DB ni logs
■ Conexión SSL	sslmode=require en la cadena de conexión Supabase. Cifrado en tránsito garantizado
■ .env en .gitignore	Creado y verificado (git check-ignore .env) ANTES del primer commit del repositorio
■ Secretos en Render	DATABASE_URL definida solo en Render Environment (sync: false en render.yaml)
■ Control de roles	Middleware en app.py: supervisores solo ven/editan su grupo — filtro por GRUPO_USER
■ .env.example	Documenta variables necesarias con valores de ejemplo, nunca credenciales reales
■ ■ NullPool SQLAlchemy	Previene conexiones persistentes que causarían errores en entorno serverless Render

Concurrencia Multi-Supervisor — Sin conflictos

Actor	Operación DB	Alcance	Conflicto
Coordinador (carga inicial)	TRUNCATE tareo INSERT INTO tareo	Todos los grupos	No — operación inicial única
Supervisor Gr. N (edita condiciones)	UPDATE tareo WHERE id IN (filas_N)	Solo grupo N	No — no toca otros grupos
Supervisor Gr. E (edita condiciones)	UPDATE tareo WHERE id IN (filas_E)	Solo grupo E	No — no toca otros grupos
Supervisor Gr. 1...5 (aprueba)	UPDATE tareo SET Aprobado=TRUE WHERE...	Solo su grupo	No — UPDATE atómico por grupo
Coordinador (reporte final)	SELECT * FROM tareo WHERE Aprobado=TRUE	Todos los grupos	No — solo lectura

Despliegue en Render — Producción

Parámetro	Valor
URL pública	https://operaciones-ysvc.onrender.com

Blueprint	render.yaml (en repositorio) — gestiona build + start + env
Rama seguida	main (auto-deploy en cada push)
Runtime	Python 3.12.7 (.python-version)
Build command	pip install -r requirements.txt
Start command	streamlit run app.py --server.port \$PORT --server.address 0.0.0.0 --server.headless true
DATABASE_URL	Variable secreta en Render Environment (sync: false — nunca en repo)
Plan	Free (1 instancia web, sleep por inactividad > 15 min)

Dashboard — 7 Gráficos Interactivos (Plotly)

#	Gráfico	Tipo	Datos mostrados
A1	Distribución de Turnos	Donut	% DÍA vs NOCHE · etiquetas con conteo y porcentaje
A2	Personas por Grupo y Turno	Barras apiladas	Conteo DÍA+NOCHE por cada grupo de trabajo
A3	Condiciones de Jornada	Pie	Normal / Corrido (C) / Teórico 12h · % de cada uno
A4	Prom. Horas Marcación por Grupo	Barras horiz.	Promedio de horas brutas por grupo · escala de color
B1	TTHH Desglosado por Grupo	Barras apiladas	Hora normal + hora 25% + hora 35% acumulado
B2	Distribución Total Horas Extra	Donut	% Normal vs 25% vs 35% sobre el total de TTHH
B3	Estado de Aprobación por Grupo	Barras horiz.	Aprobados (verde) vs Pendientes (gris) por grupo

 **Filtros interactivos:**

Multiselect grupos (coordinador ve todos, supervisor solo el suyo) · Turno: DÍA / NOCHE / Todos · KPIs: Registros · TTHH Total · Hora 25% · Hora 35% · % Aprobado

Módulo Compartir por WhatsApp

Tipo de mensaje	Contenido	Roles	Método
Resumen del Tareo	Total registros · Turnos DÍA/NOCHE · Corrido/Teórico · TTHH total · Hora 25% vs 35% · Detalle por grupo	Coordinador	Botón verde → wa.me URL
Reporte Operaciones	Total aprobados · Horas por tramo · Detalle por área/grupo con TTHH, Normal, Corrido, 25%, 35%	Coordinador, Supervisor	Botón verde → wa.me URL
Archivos Excel adjuntos	REPORTE_TAREO_SISTEMA.xlsx (tareo trabajado) · REPORTE_OPERACIONES.xlsx (operaciones)	Coordinador, Supervisor	Botón verde → adjuntar en WhatsApp
Gráficos PNG	Cualquier gráfico del Dashboard exportable con el ícono  de la barra de herramientas de Plotly (escala PNG para alta resolución)	Coordinador, Supervisor	Botón verde → adjuntar en WhatsApp

Archivos del Proyecto

Archivo	Líneas	Descripción
app.py	~780	Dashboard Streamlit — 6 pestañas, login, sidebar, estado
tareo_core.py	~567	Motor RR.HH. — cálculo TTHH, tramos, exportar Excel/CSV
dashboard_charts.py	~230	7 gráficos Plotly + helpers texto/URL WhatsApp
db.py	~270	Capa Supabase — engine, init_schema, CRUD tareo/config/usuarios
auth.py	~145	Autenticación SHA-256 + salt, crear/actualizar/eliminar usuarios
generar_requerimiento_murdoch.py	516	Generador PDF requerimiento integración Murdoch Sistemas
render.yaml	16	Blueprint Render — tipo web, build/start, envVars
requirements.txt	7	streamlit · pandas · plotly · openpyxl · SQLAlchemy · psycopg2
.env.example	6	Template DATABASE_URL sin valores reales (seguridad)
REPORTE_TAREO_SISTEMA.xlsx	—	Archivo de ejemplo — 160 registros de asistencia
REPORTE_OPERACIONES.xlsx	—	Archivo de referencia — reporte corporativo validado